

Algorithmique et Structures de Données 1

Travaux Pratiques 1

TP 1 : complexité, pointeurs, allocation dynamique

Le travail en séance de Travaux Pratiques s'effectue dans l'environnement Linux avec le compilateur Gnu g++. Vous pouvez utiliser d'autres compilateur/système d'exploitation/environnement de développement sur d'autres machines ou sur celles des salles de TP, par exemple Visual Studio Code. **Vous travaillerez** autant que possible **en binômes**.

Partie 0. Rappel

Votre espace de travail comporte une fenêtre de terminal pour lancer les commandes de compilation et d'exécution, une fenêtre d'éditeur de texte pour écrire le code, une fenêtre avec le sujet du TP disponible.

(i) **Compilation**

Téléchargez l'archive TP1.zip sur Madoc, extrayez son contenu et placez-le dans un répertoire adéquat de la hiérarchie de vos documents (par exemple ~/X21I010-ASD1/TP1/). Dans le terminal, utilisez les commandes de changement de répertoire courant (e.g. `cd X21I010-ASD1/TP1`) pour accéder à ce répertoire et lancez la compilation du fichier `progCorrect.cpp` (la commande est, par exemple,

```
gpp -std=c++11 algo1.cpp -o algo1
```

La compilation devrait bien se passer : pas de message d'erreur. Demandez alors l'exécution du code machine obtenu avec la commande

```
./algo1
```

Partie 1. Complexité

Vous trouverez 5 algorithmes dans l'archive TP1.zip, compilez-les et testez l'exécution. Il s'agit de 5 algorithmes, chacun prenant en paramètre un entier qui influencera le nombre de tour de l'algorithme. Par exemple, algorithme 2 prend en entrée la taille d'un tableau de N entiers consécutifs et ensuite effectue la recherche dichotomique d'un entier aléatoire:

```
>./algo2 10000
```

```
Found 9392 in the array of 10000 elements in 14 steps
```

Votre but est, pour chaque algorithme, de :

- 1) construire un tableau de relation entre le paramètre d'entrée N et le nombre des étapes rapportée par l'algorithme pour arriver à la solution;
- 2) tracer un graphique de nombre des étapes en fonction de N pour chaque algorithme. Que pouvez-vous dire sur le comportement de ces courbes ? Classez leur comportement dans les classes: exponentielle, constante, polynomiale, logarithmique, linéaire;
- 3) mesurer le temps d'exécution de chaque algorithme pour des différents N. Vous pouvez le faire en rajoutant la commande `time` avant chaque commande d'exécution, par exemple

```
time ./algo2 10
```

Analysez le résultat.

Partie 2. Pointeurs et allocation dynamique

1. Crazy

- Simuler l'exécution d'algorithme ci-dessous et écrire (sur papier) les états de chaque variable lors de l'exécution.
- Recopier l'algorithme en C++ et comparer la sortie avec votre réponse.

```
int main ( )
{
    int a = 5, b = 7, c = 9;
    int *p1, *p2;
    p1 = &a;
    p2 = &c;
    *p1 = ( *p2 )++;
    p1 = p2;
    p2 = &b;
    *p1 -= *p2;
    ++( *p2 );
    *p1 *= *p2;
    a = ( ++( *p2 ) ) * *p1;
    p1 = &a;
    *p2 = *p1 /= *p2 ;
    return EXIT_SUCCESS;
}
```

2. Allocation dynamique

Ecrire un programme effectuant la suite d'opérations suivantes :

- Allocation dynamique de mémoire pour un tableau d'entiers d'une taille donnée par l'utilisateur;
- Demande des valeurs des cases du tableau à l'utilisateur;
- iCréation dynamique d'un nouveau tableau destiné à recevoir les carrés des valeurs contenues dans le premier, et calcul des valeurs de ce tableau;
- Suppression du premier tableau, affichage des valeurs du second, puis suppression du second.

3. Tic-tac-toe++

Votre but est d'écrire un jeu tic-tac-toe ... où la taille de la grille carrée est passé en paramètre au début de jeu ! Pour la grille de taille $N \times N$, le joueur qui construit une suite de ses symboles ('X' ou 'O' ou autre) en ligne/colonne/diagonale de taille N gagne.

Le jeu se déroulera de la façon suivante:

- Demander les utilisateur de choisir la taille de la grille.
- Demander le symbole que le joueur 1 utilisera. Ensuite, celui de joueur 2.
- Afficher la grille vide et proposer à, par exemple, joueur 1 de jouer en saisissant les coordonnées de la ligne et cologne où il/elle veut placer leur symbole. Re-afficher la grille. Proposer à joueur 2 à jouer. Re-afficher la grille.
- Décider s'il y a un(e) gagnant(e). Sinon, continuer au tour suivant.

Vous utiliserez un tableau dynamique à deux dimensions de type `char` pour représenter la grille.

Vous allez avoir besoin de plusieurs fonctions. Par exemple,

- Une fonction de construction d'une grille vide
- Une fonction d'affichage d'une grille sur l'écran de console (affichez des bords des colonnes/lignes avec des symboles | et _). Il sera utile d'afficher également les numéros des lignes et colonnes également.
- Une fonction qui, étant donné un symbole et la position, insère le symbole sur la grille. (Ici, attention aux tricheurs qui veulent re-écrire par-dessus d'un symbole existant.)
- Une fonction qui, étant donnée un symbole, la grille, et la longueur de la suite gagnante N, décide si le symbole a gagné.
- etc. etc.